Competitive Coding
Lab Manual
303105259

Department of Information technology Parul

Institute of Engineering and Technology

Faculty of Engineering and Technology

Parul University

Session 2025-26

Published by:

Faculty of Engineering and Technology

Department of Information technology

Parul University

EXPERIMENT NO. 1

Objective(s): To design and implement a specialized stack, MinStack, that supports all standard stack operations (`push`, `pop`, `top`) along with an efficient `getMin()` operation in constant time.

Outcome: Students will be able to augment a standard data structure with additional functionalities and understand the time-space complexity trade-offs involved in designing efficient algorithms.

Problem Statement: Write a program for implementing a MINSTACK which should support operations like `push`, `pop`, `top`, and `getMin` in O(1) time. The stack should handle overflow and underflow conditions (or in Java's case, throw exceptions for underflow).

1. Construct a stack.
2. Push elements.
3. Pop elements.
4. Get the Top element.
5. Retrieve the minimum element from the stack.

Background Study: In Java, the modern and preferred way to implement a stack is using the `Deque` interface with its `ArrayDeque` implementation. A standard `Deque` provides `push()`, `pop()`, and `peek()` in O(1) time. However, finding the minimum element would require iterating through all elements, taking O(n) time.

To optimize this, we use an auxiliary `Deque`, let's call it `minStack`. This stack will store the minimum element seen *so far* at each stage.

- When we `push(x)` onto the main stack, we compare `x` with the top of `minStack`. We push `Math.min(x, minStack.peek())` onto `minStack`.
- When we `pop()` from the main stack, we also `pop()` from `minStack`. This ensures that `minStack.peek()` always holds the minimum element of the entire main stack in O(1) time.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the space complexity of this `MinStack` implementation? How can it be optimized if space is a major concern?

2. Why is `ArrayDeque` preferred over the `Stack` class in modern Java?
3. What would happen if we only pushed to `minStack` when the new element is *strictly smaller* than the current minimum?
4. How would you modify this implementation to support a `getMax()` operation in O(1) time as well?
5. Can this logic be implemented using only one stack? (Hint: consider pushing pairs of `(value, currentMin)`).

EXPERIMENT NO. 2

Objective(s): To understand and implement an algorithm for evaluating postfix (Reverse Polish Notation) expressions using a stack.

Outcome: Students will be able to write a program that correctly evaluates postfix expressions and understand the role of stacks in expression evaluation and parsing.

Problem Statement: Write a program to deal with real-world situations where Stack data structure is widely used Evaluation of expression: Stacks are used to evaluate expressions, especially in languages that use postfix or prefix notation. Operators and operands are pushed onto the stack, and operations are performed based on the LIFO principle.

Background Study: Postfix notation (RPN) is a mathematical notation where every operator follows all of its operands. For example, the infix expression `(3 + 4) * 5` is written as `3 4 + 5 *` in postfix.

Stacks are the ideal data structure for evaluating RPN. The algorithm is:

1. Scan the expression from left to right.
2. If the token is an operand (a number), push it onto the stack.
3. If the token is an operator, pop the top two operands from the stack.
4. Perform the operation (note the order: `op1 = stack.pop()`, `op2 = stack.pop()`, result is `op2 op op1`) and push the result back onto the stack.
5. After scanning all tokens, the final result will be the only element on the stack.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. Why is a stack suitable for postfix evaluation but not for infix evaluation directly?
2. The provided code takes a `String[]`. How would it change if the input was a single `String` like `"231*+9-"` (assuming single-digit numbers)?
3. What are the advantages of postfix notation for compilers?
4. Convert the infix expression `(a + b) * (c - d) / e` to postfix.
5. How would you handle potential exceptions, like `ArithmeticException` (e.g., division by zero) or invalid input?

EXPERIMENT NO. 3

Objective(s): To learn and implement an efficient stack-based algorithm to find the Next Greater Element (NGE) for each element in an array.

Outcome: Students will be able to solve problems related to finding the next greater/smaller element in a sequence and understand the "monotonic stack" pattern.

Problem Statement: Write a program for finding the Next Greater Element (NGE) for each element in a given array. The NGE for an element `x` is the first greater element on the right side of `x` in the array. If no such element exists, the NGE is considered -1.

Background Study: A naive approach uses two nested loops: for each element, scan the rest of the array to its right. This is O(n^2) and too slow for competitive programming.

A more efficient O(n) approach uses a stack. This is a classic "monotonic stack" problem. We process the array from *right to left*. The stack will store elements we have already seen, maintaining a decreasing order. When processing an element `arr[i]`:

1. While the stack is not empty and the element at the top (`stack.peek()`) is less than or equal to `arr[i]`, pop it. These popped elements can never be the NGE for `arr[i]`.
2. If the stack is now empty, it means no element to the right is greater. The NGE for `arr[i]` is -1.
3. If the stack is not empty, `stack.peek()` is the first greater element to the right. This is the NGE.
4. Push `arr[i]` onto the stack for future elements (to its left) to compare against.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the time and space complexity of this stack-based approach?
2. How would you modify the algorithm to find the "Next Smaller Element"?
3. How would you find the "Previous Greater Element" for each element? (Hint: Iterate from left to right).
4. This stack is called a "monotonic stack". Why?
5. Can you solve this problem for a circular array (where the last element is "next to" the first)?

EXPERIMENT NO. 4

Objective(s): To understand the concept and implementation of a circular queue (circular buffer), a fixed-size queue that efficiently uses its storage.

Outcome: Students will be able to design and implement a fixed-size circular queue supporting `enQueue`, `deQueue`, `Front`, and `Rear` operations, and correctly handle empty and full (overflow/underflow) conditions.

Problem Statement: Write a program to design a circular queue of size `k` which should implement the below functions: a. `enQueue(value)`: Adds an item to the queue. Returns `true` on success. b. `deQueue()`: Removes an item from the queue. Returns `true` on success. c. `Front()`: Get the front item from the queue. d. `Rear()`: Get the last item from the queue. e. `isEmpty()` / `isFull()`

Background Study: A standard queue implemented with an array can waste space. When elements are dequeued, the space at the front is left empty. A circular queue solves this by treating the array as circular. The position after the last element is the first element.

We use two pointers, `head` (or `front`) and `tail` (or `rear`).

- `head` points to the index of the first element.
- `tail` points to the index of the last element.
- `enQueue`: Insert at `(tail + 1) % size`.
- `deQueue`: Remove from `head`, then move `head` to `(head + 1) % size`.

Managing empty/full states can be tricky. A common method is to store the `size` (capacity) and a `count` of current elements.

- **Empty:** `count == 0`
- **Full:** `count == size` This avoids the ambiguity of `head == tail`.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the main advantage of a circular queue over a linear queue implemented with an array?
2. Explain the role of the modulo (`%`) operator. What would happen without it?

3. An alternative implementation doesn't use `count`. How does it check `isFull()`? (Hint: `(tail + 1) % capacity == head`). What is a disadvantage of that method?
4. Can you implement a queue using two stacks? What would be the (amortized) time complexity of its operations?
5. What are some real-world applications of a circular buffer?

EXPERIMENT NO. 5

Objective(s): To learn the Shunting-yard algorithm for converting infix expressions to postfix (RPN) and to implement it using a stack.

Outcome: Students will be able to write a program that translates a standard infix mathematical expression into its postfix equivalent, a crucial step for computer evaluation.

Problem Statement: Write a program for a given infix expression, and convert it to postfix notation.

Background Study: Infix notation (e.g., `a + b * c`) is ambiguous without operator precedence rules. Postfix notation (`a b c * +`) is unambiguous and easily evaluated with a stack. The Shunting-yard algorithm converts infix to postfix.

It uses a stack to hold operators.

1. **Operand:** Read an operand, append it to the output string.
2. **`(`:** Read `(`, push it onto the operator stack.
3. **`)`:** Read `)`, pop operators from the stack and append to output, until a `(` is seen. Pop and discard the `(`.
4. **Operator:** Read an operator `op`. While the stack is not empty, and the top is not `(`, and the top has precedence *greater than or equal to* `op`, pop from stack to output. Then, push `op` onto the stack.
5. **End:** When the input is finished, pop all remaining operators from the stack to the output.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. Why is a stack the right data structure for this algorithm?
2. Trace the conversion of `A * B + C / D` with the algorithm.
3. How does the algorithm handle operator associativity? (e.g., `^` is right-associative, but `+` is left-associative. Our code assumes left-associative for all).
4. What modifications would be needed to convert infix to prefix (Polish Notation)?
5. How would you handle multi-character operands (e.g., `21 + 10`)?

EXPERIMENT NO. 6

Objective(s): To learn how to use a `PriorityQueue` (heap) to efficiently solve "top-k" problems, such as finding the k-largest elements in a collection.

Outcome: Students will be able to implement solutions using `PriorityQueue` and understand its O(n log k) performance benefit over an O(n log n) sorting approach.

Problem Statement: Write a program for finding the Product of the three largest Distinct Elements. Use a Priority Queue to efficiently find and remove the largest elements.

Background Study: A `PriorityQueue` in Java is a *min-heap* by default (smallest element at the top). To get a *max-heap* (largest element at the top), we provide a reverse comparator: `new PriorityQueue<>(Collections.reverseOrder())`.

To solve this problem:

1. Handle "distinct" elements by first adding all numbers to a `HashSet`.
2. Create a max-heap (`PriorityQueue`) from the distinct elements.
3. `poll()` (remove) the top three elements. These will be the three largest.
4. Calculate their product.

An alternative O(n log k) approach for k=3 (used when `n` is very large):

1. Create a *min-heap* of size 3.
2. Iterate through the array. For each `num`, add it to the heap.
3. If the heap's size exceeds 3, `poll()` the smallest element.
4. After the loop, the heap contains the 3 largest elements.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the time complexity of this solution? Compare it to a solution that first sorts the array (e.g., `Arrays.sort()`).
2. How would you change the code to find the product of the three *smallest* distinct elements?
3. By default, `new PriorityQueue<>()` is a min-heap. How does `Collections.reverseOrder()` change this?
4. What is the "top-k" approach (O(n log k)) described in the background? Implement it.

5. If the input array could contain negative numbers, finding the "largest product" is harder (e.g., `-10 * -10 * 1` vs `5 * 6 * 7`). How would you solve that?

EXPERIMENT NO. 7

Objective(s): To understand the structure of linked lists and implement the merge operation for two sorted linked lists to create a single sorted list.

Outcome: Students will be able to manipulate object references in linked lists and implement an efficient in-place merge algorithm.

Problem Statement: Write a program to merge two sorted linked lists into a single sorted linked list.

Background Study: Merging two sorted lists is a fundamental linked list operation. The goal is to combine the nodes of two lists (which are already sorted) into a single sorted list.

The iterative approach is efficient and uses O(1) space. It involves:

1. Creating a `dummy` head node. This simplifies the logic, as we don't need a special case for the head of the new list.
2. Maintaining a `tail` pointer that always points to the last node in the merged list so far.
3. Comparing the heads of the two input lists (`list1` and `list2`).
4. Appending the smaller of the two nodes to `tail.next`, and advancing the pointer of the list from which the node was taken.
5. Moving `tail` to the node we just added.
6. When one list is exhausted, append the remainder of the other list to `tail.next`.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the time and space complexity of this iterative merge algorithm?
2. How would you implement the merge function recursively? What are the trade-offs (e.g., space complexity)?
3. Why is using a `dummy` head node a good practice in this problem? What edge cases does it simplify?
4. This algorithm is the core component of which "divide and conquer" sorting algorithm?
5. What happens if one or both of the input lists are empty? Does the code handle this correctly?

EXPERIMENT NO. 8

Objective(s): To solve the problem of finding the intersection point of two linked lists using an efficient O(1) space and O(m+n) time approach.

Outcome: Students will be able to implement two different optimal solutions for finding a list intersection, reinforcing their understanding of list traversal and pointer manipulation.

Problem Statement: Write a program to find the merge point (intersection node) of two linked lists. If they do not intersect, return null.

Background Study: Two lists intersect if they share a common node. From that node, their structures are identical.

- **Naive (O(m*n)):** Nested loops.
- **HashSet (O(m+n) time, O(m) space):** Add all nodes from list A to a `HashSet`. Iterate through list B and check if any node is in the set.
- **Optimal 1 (O(m+n) time, O(1) space):**
    1. Find the lengths of both lists, `lenA` and `lenB`.
    2. Calculate the difference, `diff = abs(lenA - lenB)`.
    3. Move the head pointer of the *longer* list forward by `diff` nodes.
    4. Now, both lists have an equal number of nodes remaining.
    5. Traverse both lists simultaneously. The first node where the pointers are equal (`pA == pB`) is the intersection. If they reach the end (`null`), there is no intersection.
- **Optimal 2 (O(m+n) time, O(1) space):**
    1. Use two pointers, `pA` and `pB`, starting at `headA` and `headB`.
    2. Traverse both. If `pA` reaches the end, redirect it to `headB`.
    3. If `pB` reaches the end, redirect it to `headA`.
    4. This "switching" effectively makes both pointers travel `lenA + lenB` (if no intersection) or `lenA + lenB_prefix` (if intersection). They will meet at the intersection point.

Algorithm (Student Work Area - Optimal 1):

Code (Student Work Area - Optimal 1):

Question Bank:

1. Why must the intersection point and all subsequent nodes be shared? (Hint: `next` reference).
2. Describe the "two-pointer switch" (Optimal 2) algorithm. Why does it work?

3. What happens in this algorithm if the lists do not intersect?
4. Why is the `HashSet` method O(m) in space, while this one is O(1)?
5. What is being compared in `pA != pB`? The values (`.val`) or the object references? Why is this important?

EXPERIMENT NO. 9

Objective(s): To practice in-place modification of a linked list by swapping adjacent nodes.

Outcome: Students will be able to confidently manipulate list references to alter the structure of a linked list in-place, handling various edge cases correctly.

Problem Statement: Given a linked list, write a program to swap every two adjacent nodes and return its head. You may not modify the values in the list's nodes, only nodes (references) themselves may be changed.

Background Study: This is a classic pointer/reference manipulation problem. For a list like `1 -> 2 -> 3 -> 4`, we want `2 -> 1 -> 4 -> 3`. The challenge is rewiring the `next` references.

- The node *before* the pair must point to the *second* node.
- The *first* node must point to the node *after* the pair.
- The *second* node must point to the *first* node.

A `dummy` head node is extremely helpful here, as it acts as the "node before" the very first pair, simplifying the logic.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. Why is the dummy node so useful? What extra `if` condition would be needed without it?
2. Can this problem be solved recursively? If so, what is the recursive relation?
3. What is the time and space complexity of this iterative solution?
4. How would you modify this code to swap nodes in groups of `k=3` (e.g., `1->2->3->4->5` becomes `3->2->1->4->5`)?
5. Trace the pointers (`prev`, `first`, `second`) for the first two iterations on the list `1 -> 2 -> 3 -> 4`.

EXPERIMENT NO. 10

Objective(s): To understand the properties of a Binary Search Tree (BST) and to implement a function that validates if a given binary tree is a valid BST.

Outcome: Students will be able to write a correct and efficient validation function for BSTs, understanding the critical difference between local and global BST properties (range constraints).

Problem Statement: Write a program for building a function `isValidBST` to validate if a given binary tree is a Binary Search Tree.

Background Study: A Binary Search Tree (BST) has a strict ordering property:

1. The left subtree of a node contains only nodes with keys *less than* the node's key.
2. The right subtree of a node contains only nodes with keys *greater than* the node's key.
3. Both the left and right subtrees must also be valid BSTs.

A common mistake is to only check `node.left.val < node.val` and `node.right.val > node.val`. This is *wrong*. The property must hold for all descendants. For example, a right child of a root `5` cannot have a left child of `4`.

The correct approach is to check that each node's value falls within a valid *range* `(min, max)`.

- The root's range is `(null, null)` (or `(-inf, +inf)`).
- When we go to a *left* child, we update the *max* of the range to its parent's value: `(parent.min, parent.val)`.
- When we go to a *right* child, we update the *min* of the range to its parent's value: `(parent.val, parent.max)`.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. Why does the naive approach (checking only `node.left.val < node.val`) fail? Provide a tree structure where it would incorrectly return `true`.
2. An alternative validation method is to perform an In-order traversal and check if the resulting list is strictly increasing. Explain why this works.

3. What is the time and space complexity of the recursive range-based validation?
4. Why does the code use the `Long` object wrapper class instead of the `long` primitive?
5. What are the advantages of a *balanced* BST (like an AVL or Red-Black Tree) over a simple BST?

EXPERIMENT NO. 11

Objective(s): To understand and implement the insertion operation in a Binary Search Tree (BST).

Outcome: Students will be able to write a function that correctly inserts new values into a BST while maintaining its properties, and use this function to build a tree from a sequence of numbers.

Problem Statement: Write a program to build a Binary Search Tree (BST) by implementing the `insert` function.

Background Study: A BST is built by inserting nodes one by one. The `insert` operation is fundamental. To insert a new value `val` into a BST:

1. If the `root` is `null`, the tree is empty. Create a new `TreeNode` with `val` and return it as the new root.
2. If `val` is less than `root.val`, the new node belongs in the left subtree. Recursively call `insert` on the left child: `root.left = insert(root.left, val)`.
3. If `val` is greater than `root.val`, the new node belongs in the right subtree. Recursively call `insert` on the right child: `root.right = insert(root.right, val)`.
4. If `val` is equal to `root.val`, we can either ignore the duplicate (common in CP) or handle it (e.g., store a count). We will ignore it.
5. Return the (possibly modified) `root` reference.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the time complexity of inserting a single element into a BST?
2. What is the worst-case time complexity for building a BST from `n` elements? What kind of input causes this? (e.g., a sorted array)
3. How does the order of insertion affect the final *shape* of the BST?
4. How would you implement the `insert` function iteratively instead of recursively?
5. What happens if you try to insert a duplicate value with the code provided? How would you modify it to store a *count* of duplicate elements?

EXPERIMENT NO. 12

Objective(s): To implement a function to find the maximum depth (or height) of a binary tree using recursion.

Outcome: Students will be able to solve tree problems using Depth-First Search (DFS) and understand how to formulate a simple recursive solution for tree properties.

Problem Statement: Write a program to determine the depth of a given Tree by implementing `MAXDEPTH`.

Background Study: The maximum depth of a binary tree is the number of nodes along the longest path from the root node down to the farthest leaf node. This is a classic recursive problem. The logic is:

- The depth of an empty tree (a `null` node) is 0.
- The depth of a non-empty tree is `1` (for the current node) plus the *maximum* of the depths of its left and right subtrees.
- `maxDepth(root) = 1 + Math.max(maxDepth(root.left), maxDepth(root.right))`

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the difference between the "depth" of a node and the "height" of a tree?
2. What is the time complexity of this `maxDepth` function?
3. How could you solve this problem iteratively using Breadth-First Search (Level Order Traversal) with a `Queue`?
4. What is the *minimum* depth of a binary tree? How would you modify this function to find that?
5. How is the space complexity of this recursive solution determined? (Hint: recursion call stack).

EXPERIMENT NO. 13

Objective(s): To understand and implement the three fundamental Depth-First Search (DFS) traversals for a binary tree: Pre-order, In-order, and Post-order.

Outcome: Students will be able to traverse a tree using all three DFS methods and understand how the order of visiting the Root (N), Left (L), and Right (R) subtrees defines each traversal.

Problem Statement: Write a program to understand and implement Tree traversals: Pre-Order, Post-Order, and In-Order.

Background Study: Tree traversals are methods for visiting every node in a tree. The three main DFS traversals are:

1. **In-order (L-N-R):** Visit left subtree, then the node, then the right subtree. Used to retrieve elements from a BST in sorted order.
2. **Pre-order (N-L-R):** Visit the node, then the left subtree, then the right subtree. Used for copying a tree or prefix notation.
3. **Post-order (L-R-N):** Visit the left subtree, then the right subtree, then the node. Used for deleting a tree (deleting children before parent).

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. If you are given the In-order and Pre-order traversals of a tree, can you reconstruct the unique tree? Why?
2. What is the output of an In-order traversal on a valid Binary Search Tree?
3. Which traversal is used to safely delete all nodes in a tree, and why?
4. How would you implement an iterative In-order traversal? (Hint: requires a `Deque` acting as a stack).
5. What is Level-order (BFS) traversal, and what data structure does it use?

EXPERIMENT NO. 14

Objective(s): To implement the boundary traversal of a binary tree, a non-standard traversal that combines "left view," "leaves," and "right view" in an anti-clockwise direction.

Outcome: Students will be able to combine multiple traversal algorithms and handle edge cases (like leaf nodes vs. boundary nodes) to solve complex tree problems.

Problem Statement: Write a program to perform Boundary Traversal on a Binary Tree. (The syllabus says BST, but this is a standard problem for any Binary Tree and does not use BST properties).

Background Study: The boundary traversal prints the anti-clockwise "outline" of the tree. It consists of three parts:

1. **The Left Boundary:** All nodes on the left edge, from the root down to the last non-leaf node.
2. **The Leaf Nodes:** All leaf nodes, printed from left to right.
3. **The Right Boundary:** All nodes on the right edge, from the last non-leaf node *up to* the root (printed in reverse).

Care must be taken to not print a node (especially the root) or a leaf node multiple times.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. Why is the root node handled as a special case before all other calls?
2. What happens if the tree is heavily skewed to the left? Trace the traversal.
3. Why is the right boundary printed *after* the recursive call, while the left is printed *before*?
4. Why do we need to check `isLeaf` in all three helper functions?
5. What would happen if we just called `printLeaves(root.left)` and `printLeaves(root.right)` without the root `isLeaf` check?

EXPERIMENT NO. 15

Objective(s): To find the Lowest Common Ancestor (LCA) of two given nodes in a Binary Search Tree (BST) efficiently.

Outcome: Students will be able to write an efficient O(H) function (where H is the tree height) that utilizes the unique ordering properties of a BST to find the LCA.

Problem Statement: Write a program for finding the Lowest Common Ancestor (LCA) in a Binary Search Tree.

Background Study: The Lowest Common Ancestor (LCA) of two nodes, `p` and `q`, in a BST is the lowest node that has both `p` and `q` as descendants. The BST's ordering property allows for a very fast search.

Start at the root. For any `currentNode`:

1. If `p.val` and `q.val` are both *smaller* than `currentNode.val`, their LCA must be in the left subtree.
2. If `p.val` and `q.val` are both *larger* than `currentNode.val`, their LCA must be in the right subtree.
3. If the values "split" (one is smaller, one is larger), or if one of them *is* the `currentNode`, then the `currentNode` is the split point and must be the LCA.

This can be implemented iteratively (O(1) space) or recursively.

Algorithm (Student Work Area - Iterative):

Code (Student Work Area):

Question Bank:

1. What is the time complexity of this algorithm for a balanced BST? What about an unbalanced (skewed) BST?
2. What is the space complexity of this iterative solution? How does it compare to a recursive one?
3. How would the algorithm change if this were a general Binary Tree, *not* a BST? (This is a much harder problem).
4. What happens if one of the nodes (e.g., `p`) is the LCA of `p` and `q`? Does the code handle this?
5. What assumptions does this code make about the nodes `p` and `q`? (e.g., do they have to exist in the tree?)

EXPERIMENT NO. 16

Objective(s): To determine if a binary tree is symmetric (a mirror image of itself).

Outcome: Students will be able to write a recursive function that compares two subtrees *simultaneously* to validate a structural property.

Problem Statement: Write a program to verify and validate if a tree is mirrored (symmetric) or not.

Background Study: A tree is symmetric if its left subtree is a mirror image of its right subtree. This means:

1. The root's left child's value must equal the root's right child's value.
2. The left child's *left* subtree must be a mirror of the right child's *right* subtree.
3. The left child's *right* subtree must be a mirror of the right child's *left* subtree.

This suggests a recursive helper function that takes two nodes, `t1` and `t2`, and checks if they are mirror images of each other.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. Why can't we just check if the left and right subtrees are identical (i.e., `isSameTree(root.left, root.right)`)?
2. What is the time complexity of this solution in terms of `n` (number of nodes)?
3. Can this be solved iteratively? (Hint: Use two `Queues` or `Deques` for a level-by-level comparison).
4. What is the difference between this problem ("Symmetric Tree") and the "Same Tree" problem?
5. Trace the algorithm on a simple symmetric tree with 3 levels.

EXPERIMENT NO. 17

Objective(s): To understand the core concept of a hash function and demonstrate its use in a simple hash table.

Outcome: Students will be able to implement a basic hash function and understand its role in mapping keys to array indices for O(1) average-case lookups.

Problem Statement: Write a program for a basic hash function. Demonstrate its usage to store and retrieve key-value pairs.

Background Study: A hash function is a function that takes an input (or 'key') and returns an integer. This integer is used as an index (a "hash code") into an array, which is the "hash table." A good hash function distributes keys uniformly across the array to minimize collisions.

A very simple hash function for a string is to sum the ASCII values of its characters and then use the modulo operator (`%`) with the table size to ensure the index is within the array bounds. `hash = (sum_of_chars) % table_size`

This is not a great hash function (e.g., "abc" and "cba" hash to the same index), but it demonstrates the concept. We will store pairs in an array to show the mapping.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is a hash collision? What happens in the code above when "name" and "mean" are inserted?
2. Why is the modulo (`%`) operator essential in a hash function?
3. Is this a good hash function? Why or why not? (Hint: "abc" and "cba" hash to the same value).
4. How can this hash function be improved? (e.g., `hash = (hash * 31 + c)`).
5. What are the key properties of an ideal hash function?

EXPERIMENT NO. 18

Objective(s): To implement a functional hash table using "separate chaining" to correctly handle collisions.

Outcome: Students will be able to build a hash table class that supports insertion, deletion, and searching, with a robust collision resolution strategy.

Problem Statement: Implement a hash table using separate chaining for collision handling. Perform operations like insertion, deletion, and search on the hash table.

Background Study: Separate chaining is a collision resolution technique where the hash table's array does not store the values directly. Instead, each array index (or "bucket") holds a pointer to a data structure, typically a `LinkedList`, which stores all the key-value pairs that hash to that same index.

- **Insertion:** Hash the key to find the bucket. Traverse the `LinkedList` in that bucket. If the key exists, update its value. If not, add a new `(key, value)` pair to the list.
- **Search:** Hash the key. Traverse the list in the corresponding bucket. If the key is found, return its value.
- **Deletion:** Hash the key. Traverse the list to find the key. Remove the `(key, value)` pair from the list.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the main advantage of separate chaining over the simple method in Experiment 17?
2. What is the worst-case time complexity for `insert`, `search`, and `remove`? When does this occur? (Hint: all keys hash to one bucket).
3. What is the "load factor" of a hash table? How does it relate to "rehashing"?
4. What is the main alternative to separate chaining? (Hint: Open Addressing / Probing).
5. Why is `key.hashCode()` used? What guarantees does it provide (or not provide)?

EXPERIMENT NO. 19

Objective(s): To solve the classic "Two Sum" problem efficiently using a `HashMap`.

Outcome: Students will understand how a `HashMap` can be used to optimize a search problem from a naive O(n^2) to a linear O(n) time complexity by trading space for time.

Problem Statement: Write a program to implement the "Two Sum" problem using a HASHMAP. Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

Background Study: The naive O(n^2) solution uses two nested loops, checking every pair. A `HashMap` (which is Java's built-in hash table) allows us to do this in a single pass.

We iterate through the array once. For each element `nums[i]`, we calculate its "complement" (`complement = target - nums[i]`). This is the number we need to find.

- We check if this `complement` *already exists* as a key in our `HashMap`.
- If it does: We found our pair! We return the index of the complement (which we stored as the map's value) and the current index `i`.
- If it does not: We add the *current* number `nums[i]` (as the key) and its index `i` (as the value) to the map, so it can be found as a complement by a future element.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. Why is this one-pass `HashMap` solution O(n)? What is the (average) time complexity of a `HashMap` lookup?
2. What is the space complexity of this solution?
3. Why do we add `map.put(num, i)` *after* checking for the complement, and not before?
4. What would happen if the array contained duplicate numbers? Does this solution handle it correctly? (e.g., `nums = {3, 3}`, `target = 6`)
5. How would you solve this problem if the array was already sorted? (Hint: "Two Pointers" technique, O(n) time and O(1) space).

EXPERIMENT NO. 20

Objective(s): To understand the Trie (prefix tree) data structure and implement its core operations: `insert`, `search`, and `remove`.

Outcome: Students will be able to build a functional Trie class and understand its application for efficient prefix-based string operations, like auto-complete.

Problem Statement: Write a program to implement `search`, `insert`, and `remove` operations in a Trie.

Background Study: A Trie (from "re**trie**val") is a tree-like data structure that stores a dynamic set of strings.

- Each node represents a character.
- The path from the root to a node represents a prefix.
- Each node contains an array (or `Map`) of pointers to its children, one for each possible character (e.g., 26 for 'a'-'z').
- A boolean flag, `isEndOfWord`, at a node marks if the path to it forms a complete word.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the time complexity of `insert` and `search` in a Trie, in terms of word length `L`?
2. What is the space complexity of a Trie in the worst case?
3. What are the main advantages of a Trie over a `HashSet<String>`?
4. How would you modify the `search` function to implement a `startsWith(String prefix)` function?
5. Why is the `remove` function so much more complex than `insert` or `search`? Explain the purpose of the post-recursion check.

EXPERIMENT NO. 21

Objective(s): To understand and implement Huffman's greedy algorithm to generate prefix-codes for lossless data compression.

Outcome: Students will be able to build a Huffman Tree using a `PriorityQueue` and traverse it to generate the corresponding Huffman codes for each character.

Problem Statement: Write a program to implement Huffman coding. Given a string, first find the frequency of each character, then build the Huffman tree and print the compression codes for each character.

Background Study: Huffman coding is a greedy algorithm that assigns variable-length prefix-codes to characters based on their frequency. More frequent characters get shorter codes. It's a "prefix-code" because no code is a prefix of another, ensuring unambiguous decoding.

The algorithm uses a min-priority queue to build a binary tree (the Huffman Tree):

1. **Node:** Create a `HuffmanNode` class that implements `Comparable` (to be used in the `PriorityQueue`). It stores `char`, `frequency`, `left`, and `right` child.
2. **Frequency:** Count the frequency of each character in the input string (e.g., using a `HashMap`).
3. **Heap:** For each character, create a leaf `HuffmanNode` and add it to a `PriorityQueue<HuffmanNode>`. The `PriorityQueue` will be a min-heap, ordered by frequency.
4. **Build Tree:** While the `PriorityQueue` has more than one node: a. `poll()` the two nodes with the lowest frequencies (`left` and `right`). b. Create a new *internal* `parent` node with `frequency = left.frequency + right.frequency`. c. Set `parent.left = left` and `parent.right = right`. d. `add()` the new `parent` node back into the `PriorityQueue`.
5. The last node remaining in the queue is the `root` of the Huffman Tree.
6. **Generate Codes:** Traverse the tree from the root using DFS. Pass a `String` (or `StringBuilder`) for the current code. Go left, append "0". Go right, append "1". When a leaf node is reached, store the code for that character.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. Why is Huffman coding a "greedy" algorithm?
2. What is a "prefix-code" and why is it important for decoding?
3. Why do we use a min-priority queue for this problem? What would happen if we used a max-queue?
4. Given the codes, how would you decode a compressed string (e.g., "1011001")?
5. What is the time complexity of building the Huffman Tree?

EXPERIMENT NO. 22

Objective(s): To implement an algorithm to find the number of distinct (unique) substrings in a given string.

Outcome: Students will learn a simple O(n^2) method for finding all distinct substrings using a `HashSet` and understand the distinction between substrings and subsequences.

Problem Statement: Write a program to find the total number of distinct substrings in a string.

Background Study: A substring is a *contiguous* sequence of characters within a string. For example, the substrings of "abc" are "a", "b", "c", "ab", "bc", "abc". A subsequence is not necessarily contiguous (e.g., "ac" is a subsequence of "abc").

A string of length `n` has `n*(n+1)/2` possible substrings. However, many of these can be duplicates (e.g., "banana" has many "a" substrings).

A simple O(n^2) approach is to generate all possible substrings and store them in a `HashSet`. The `HashSet` automatically handles duplicates.

- Use a nested loop. The outer loop `i` defines the start of the substring.
- The inner loop `j` defines the end of the substring.
- `s.substring(i, j+1)` extracts the substring.
- Add this substring to the `HashSet`.
- The final answer is the `size()` of the `HashSet`.

Note: A more advanced O(n) solution involves building a Suffix Trie or Suffix Automaton and counting the total number of unique paths, but the `HashSet` method is much simpler to implement.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the time complexity of this `HashSet` solution? (Be careful: `substring` can take time).
2. What is the space complexity of this solution in the worst case?
3. How is a substring different from a subsequence?
4. A string of length `n` has `n*(n+1)/2` total substrings. Why?

5. How could you use a Trie (specifically, a Suffix Trie) to solve this problem?

EXPERIMENT NO. 23

Objective(s): To augment the Trie data structure to count the total number of words stored within it.

Outcome: Students will be able to write a recursive (DFS) function to traverse a Trie and count all nodes that are marked as the end of a word.

Problem Statement: Write a program to find the number of words in a Trie. (This is interpreted as counting all unique words stored in the Trie).

Background Study: This problem builds on Experiment 20. We have a Trie data structure where complete words are marked with an `isEndOfWord = true` flag.

To find the total number of words, we need to traverse the entire Trie and count every node that has this flag set to `true`. A Depth-First Search (DFS) is a natural way to do this.

We can write a recursive helper function `countWords(TrieNode node)` that returns the total number of word-nodes in the subtree rooted at `node`.

- **Base Case:** If `node == null`, return 0.
- **Recursive Step:**
    1. Initialize `count = 0`.
    2. If `node.isEndOfWord` is `true`, set `count = 1` (this node itself is a word).
    3. Iterate through all 26 children of the current node.
    4. For each child, recursively call `countWords(child)` and add the result to `count`.
    5. Return `count`.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the time complexity of this `countWords()` function?
2. How is this different from `search(word)`?
3. How would you modify this code to count only the words that have a specific `String prefix`?
4. What change would be needed if we wanted to count *all prefixes*, not just complete words?

5. If we stored `int wordCount` in the `Trie` class and incremented it in `insert()`, what would be the pros and cons?

EXPERIMENT NO. 24

Objective(s): To implement an algorithm to find the "Left View" of a binary tree (the set of nodes visible when viewing from the left).

Outcome: Students will be able to solve tree "view" problems using either BFS (level-order) or DFS, with a clear understanding of how to track levels.

Problem Statement: Write a program to view a tree from the Left View.

Background Study: The left view of a binary tree contains the *first* node encountered at each level.

Method 1: Level Order Traversal (BFS) This is the most intuitive approach. We perform a standard level-order traversal. For each level, we add *only* the first node we encounter to our result list.

1. Use a `Queue<TreeNode>`.
2. Loop while the queue is not empty.
3. Get `levelSize = queue.size()`.
4. Loop `i` from 0 to `levelSize - 1`.
5. `TreeNode node = queue.poll();`
6. **If `i == 0`**, this is the first node on the level. Add `node.val` to the `result` list.
7. Add children (left, then right) to the queue for the next level.

Method 2: Recursive DFS (Pre-order) This is a clever approach that uses the "depth" of the recursion.

1. `dfs(node, level, resultList)`
2. If `node == null`, return.
3. **If `level == resultList.size()`**, it means this is the *first time* we are visiting this level. Since we always visit `left` before `right` (pre-order), this node *must* be the leftmost one. Add `node.val` to `resultList`.
4. Recurse on `left` first: `dfs(node.left, level + 1, resultList)`.
5. Recurse on `right`: `dfs(node.right, level + 1, resultList)`.

Algorithm (Student Work Area - BFS):

Code (Student Work Area - BFS):

Question Bank:

1. What is the time and space complexity of this BFS solution?
2. How would you implement the "Right View" of the tree?
3. Explain the DFS (recursive) approach. Why does `level == resultList.size()` work?
4. If you implemented the DFS approach but recursed on `right` before `left`, what would you get?
5. Does this algorithm work for a "complete" binary tree?

EXPERIMENT NO. 25

Objective(s): To implement the Level Order Traversal (Breadth-First Search or BFS) of a binary tree.

Outcome: Students will be able to implement BFS for a tree using a `Queue` and organize the output by level.

Problem Statement: Write a program to traverse a tree using Level Order Traversal.

Background Study: Level order traversal visits all nodes at a given depth (or "level") before moving to the next level. It is a Breadth-First Search (BFS). The data structure used to manage the "frontier" of nodes to visit is a `Queue`.

The algorithm is as follows:

1. Create a `Queue` and add the `root` node.
2. While the `Queue` is not empty:
3. Get the number of nodes currently in the queue, `levelSize`. These nodes all belong to the *current* level.
4. Create a list for the current level's values, `currentLevel`.
5. Loop `levelSize` times: a. `poll()` a `node` from the front of the queue. b. Add `node.val` to the `currentLevel` list. c. `add()` the node's non-null `left` child to the *back* of the queue. d. `add()` the node's non-null `right` child to the *back* of the queue.
6. After the inner loop, add the `currentLevel` list to the main `result` list.

Algorithm (Student Work Area):

Code (Student Work Area):

Question Bank:

1. What is the time and space complexity of level order traversal?
2. What data structure is used for BFS? What data structure is used for DFS?
3. How would you modify this code to print the "Zigzag Level Order Traversal"? (i.e., left-to-right, then right-to-left, etc.)
4. How can you use this algorithm to find the maximum depth of the tree (Experiment 12)?
5. How is this algorithm similar to the one used for the "Left View" (Experiment 24)?